Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 5

Дисциплина МДК 02.02
Стратегия ветвления Git Flow

Вариант № 2
АИС «Электронная медицинская карта»

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3

1.1 Модель ветвления и роли.....3

1.2 Жизненный цикл feature и релиза.....4

1.3 Code Review и AppSec.....4

2 ПРОВЕРКА РЕЗУЛЬТАТОВ.....5

ЗАКЛЮЧЕНИЕ.....6

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....7

1 ВЫПОЛНЕНИЕ РАБОТЫ

Цель работы — спроектировать командные роли, жизненный цикл feature и регламент Code Review в классической модели Git Flow. Работа выполнена по варианту 2 нового сборника [2].

В примерах используются вымышленные пациенты. Оформление выполнено по требованиям института [1].

1.1 Модель ветвления и роли

Выбрана классическая Git Flow с основными main и develop и временными feature, release и hotfix. Она подходит к запланированному выпуску согласованных версий ЭМК; модель не рассматривается как обязательный подход для любой команды [3].

Сведения представлены в таблице 1.

Таблица 1 — Назначение веток

Ветка	Источник	Куда сливается
main	Релиз или hotfix	Стабильные версии с тегами
develop	Основная интеграционная	Release
feature/patient-summary	develop	develop через PR
release/1.0.0	develop	main и develop
hotfix/1.0.1	main	main и develop

Источник: составлено по учебному заданию [2].

Сведения представлены в таблице 2.

Таблица 2 — Зоны ответственности

Роль	Ответственность
Developer	Feature, небольшие коммиты, проверки и PR
Tech Lead / reviewer	Проверка границ, контрактов и разрешение слияния
QA	Позитивные/негативные тесты и стабилизация release
Release manager	Тег, комплект версии и решение о выпуске

Источник: составлено по учебному заданию [2].

1.2 Жизненный цикл feature и релиза

Задача: добавить чтение краткой назначенной карты. Разработчик создаёт ветку от develop, вносит изменение, запускает тесты и открывает PR в develop. Reviewer проверяет изменение до слияния. Последовательность команд воспроизведена в локальном Git-репозитории.

```
git switch develop
git switch -c feature/patient-summary
git add src tests docs
git commit -m "Add patient summary scope"
# На GitHub: PR feature/patient-summary -> develop
git switch develop
git merge --no-ff feature/patient-summary
git switch -c release/1.0.0
# Стабилизация и проверка доступа
git switch main
git merge --no-ff release/1.0.0
git tag -a v1.0.0 -m "Release 1.0.0"
git switch develop
git merge --no-ff release/1.0.0
```

Hotfix создаётся от main, а исправление возвращается в обе основные ветки. Merge-коммиты --no-ff сохраняют границы задач в истории [4]. При незавершённом release отдельный порядок переноса hotfix согласуется до нового релиза.

```
git switch main
git switch -c hotfix/1.0.1
# Исправление, проверка и коммит
git add .
git commit -m "Fix safe access error message"
git switch main
git merge --no-ff hotfix/1.0.1
git tag -a v1.0.1 -m "Hotfix 1.0.1"
git switch develop
git merge --no-ff hotfix/1.0.1
git branch -d hotfix/1.0.1
```

1.3 Code Review и AppSec

В PR reviewer проверяет изменение контракта, права, SQL-параметры и отсутствие секретов. SAST и анализ зависимостей запускаются в feature/PR и release; обязательны проверки 401/403, назначения пациента и отсутствия медицинских данных в открытом журнале.

2 ПРОВЕРКА РЕЗУЛЬТАТОВ

В отдельном локальном репозитории выполнены ветки feature, release и hotfix, слияния и теги v1.0.0/v1.0.1. Ниже показан граф по фактическим родительским связям коммитов. Полный журнал команд и Git bundle находятся в verification.

Результат показан на рисунке 1.

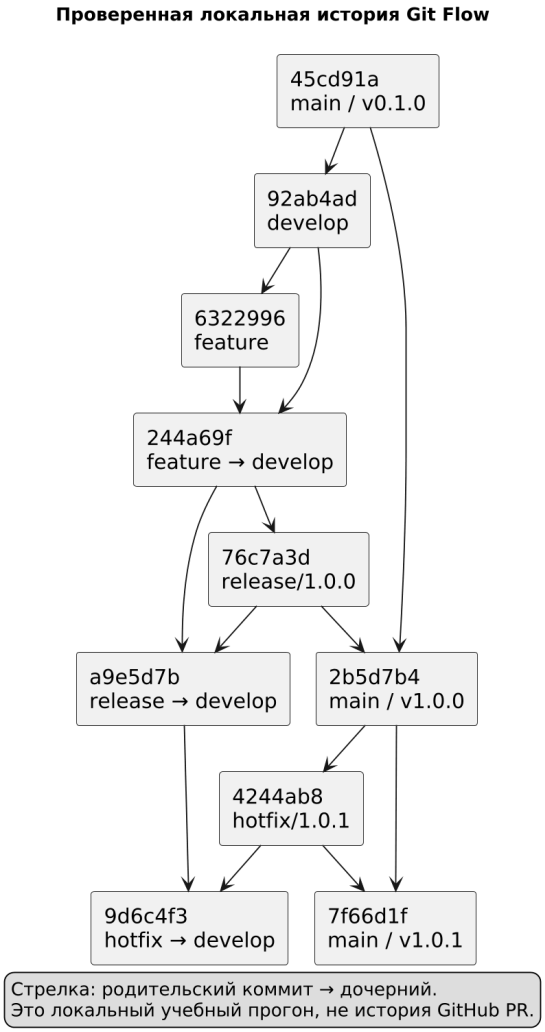


Рисунок 1 — Граф выполненного локального сценария Git Flow

Источник: схема разработана для учебного проекта.

В bundle сохранена история коммитов, слияний и тегов. Регламент независимого review и защиты main приведён в отдельной инструкции GitHub.

Исходники и материалы актуальной публикации: https://github.com/rob4ik72-del/MDK0202-EMR

ЗАКЛЮЧЕНИЕ

Спроектирован цикл изменения ЭМК от feature до релиза, включая аварийное исправление и возврат изменений в develop.

Команды проверены локально. Регламент review фиксирует ответственность за безопасность и не допускает самостоятельного одобрения своего PR.

Репозиторий проекта: https://github.com/rob4ik72-del/MDK0202-EMR

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. СПбИЭУ. Положение о требованиях по подготовке и оформлению письменных работ и приложение М: локальный нормативный акт, предоставленный в материалах обучения. [1].

2. МДК 02.02. Детализированный сборник практических работ ПР1–ПР9: электронный учебный материал; вариант 2 «Электронная медицинская карта». [2].

3. Vincent Driessen. A successful Git branching model. URL: https://nvie.com/posts/a-successful-git-branching-model/ (дата обращения: 05.11.2026) [3].

4. Git. Pro Git: Branches in a Nutshell. URL: https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell (дата обращения: 05.11.2026) [4].